

Synthèse - réponse TP

1. Contexte

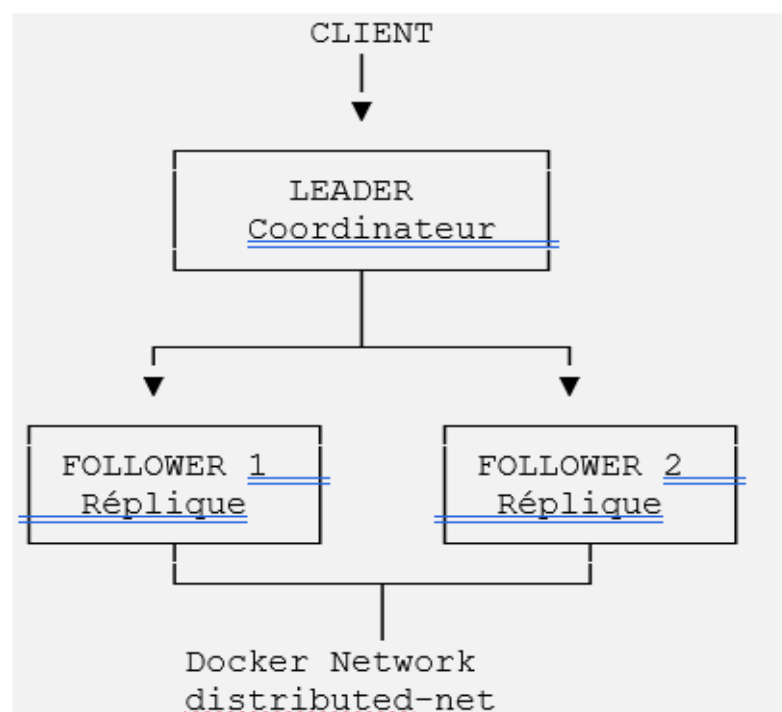
Dans un système centralisé, les données et les traitements sont regroupés sur une même machine. Quand le volume de données, le nombre de requêtes ou les exigences de disponibilité augmentent, cette architecture atteint ses limites.

L'objectif du TP est de mettre en évidence les principaux enjeux d'un système distribué : construire un mini-cluster local avec Docker, observer la communication entre plusieurs nœuds, simuler une réplication de données, puis provoquer des pannes et des partitions réseau. Kubernetes n'est pas utilisé.

2. Objectifs pédagogiques

- identifier les composants d'une architecture distribuée ;
- déployer plusieurs nœuds avec Docker ;
- comprendre la communication réseau entre les nœuds ;
- distinguer **partitionnement** et **réplication** ;
- observer le comportement d'un système lors d'une panne ;
- simuler une partition réseau ;
- identifier les difficultés de la distribution : réseau, synchronisation, cohérence et disponibilité.

3. Architecture cible



À retenir : partitionnement = répartir les données ; réplication = copier les données sur plusieurs nœuds. Les rôles Leader et Followers sont une simulation pédagogique, sans reproduire l'algorithme complet de Cassandra, Kafka ou ZooKeeper.

4. Environnement technique

- **Git** : récupérer et versionner le projet ;
- **Docker** : créer les nœuds et les conteneurs ;
- **Docker Network** : permettre la communication entre les nœuds ;
- **Docker Compose** : démarrer le mini-cluster ;
- **ligne de commande** : observer et tester le système.

5. Étape 1 — Récupération du projet

URL github : https://github.com/ThLeroy61/donnees_distrib_tp1

Objectif : comprendre le rôle de Git : il contient le code et la configuration du TP, Docker s'en sert pour créer le mini-cluster distribué.

6. Étape 2 — Création du réseau distribué

Question : Pourquoi est-il nécessaire que les différents nœuds puissent communiquer sur un réseau commun ?

Réponse :

Les nœuds ne partagent ni mémoire ni disque : la seule façon de coopérer est de s'envoyer des messages. Sans réseau commun, chaque conteneur est isolé : le Leader ne pourrait ni joindre les Followers, ni leur répliquer les données.

7. Étape 3 — Déploiement du mini-cluster

Question : Quel est le rôle de chaque nœud dans l'architecture ?

Réponse

- **node-leader** : coordinateur. Il reçoit les requêtes du client et propage les données aux Followers.
- **node-follower-1 et node-follower-2** : répliques. Ils stockent une copie des données reçues du Leader et servent de secours si un nœud tombe.

8. Étape 4 — Tester la communication entre les nœuds

Question : Comment Docker permet-il au Leader de retrouver node-follower-1 ?

Réponse : Docker intègre un DNS interne sur les réseaux créés par l'utilisateur. Il traduit le nom du conteneur (`node-follower-1`) en adresse IP : le Leader n'a pas besoin de connaître l'IP.

Question : Pourquoi la communication réseau devient-elle importante dans une architecture distribuée ?

Réponse : Tout passe par des messages. Le réseau introduit de la latence, des pertes de messages et des coupures possibles, qui n'existent pas sur une seule machine. La fiabilité du système dépend donc de la qualité de cette communication.

Question : Quelle différence avec une application utilisant uniquement localhost ?

Réponse : localhost désigne la machine (ou le conteneur) elle-même. Une application qui n'utilise que localhost tourne en un seul point, sans communication entre machines ni problème réseau. Ici, chaque nœud a sa propre identité réseau et doit être contacté à distance.

9. Étape 5 — Envoyer et visualiser des données

Les données sont envoyées avec le script `load.py` ajouté au projet : il récupère **20 relevés horaires de température à Paris** (API Open-Meteo) et les envoie en `POST` sur `http://localhost:8080/data` au nœud d'entrée, qui les réplique sur les autres nœuds.

Chaque donnée est un couple **clé = horodatage / valeur = température** (ex. `"2026-10-05T14:00"` → `"14.2 °C"`).

Chemin suivi : API Open-Meteo → client (`load.py`) → nœud d'entrée → Follower 1 et Follower 2.

Observation

Donnée (clé = horodatage)	Leader	Follower 1	Follower 2
1er relevé	✓	✓	✓
...	✓	✓	✓
20e relevé	✓	✓	✓

Observation attendue

Le script imprime 20 réponses (une par `POST`). Seul le nœud d'entrée reçoit les données du client : les deux autres les reçoivent par **réplication**. La présence des clés se vérifie sur chaque nœud avec l'endpoint de lecture de l'application. Le

résultat exact dépend de l'application fournie ; l'objectif est d'observer le principe de réplication.

10. Étape 6 — Simulation d'une panne de nœud

Ce qu'on observe	Explication
Les relevés sont sur le nœud d'entrée et le Follower 2, mais pas sur le Follower 1	Le nœud d'entrée ne peut répliquer qu'aux nœuds joignables.
Si l'application renvoie une erreur ou un timeout, le script s'arrête au relevé en cours	`load.py` n'a pas de `try/except` : une exception de `urlopen` interrompt toute la boucle. C'est une conséquence côté client.
Après `docker start node-follower-1`, il lui manque les relevés envoyés pendant la panne	Pas de resynchronisation automatique, sauf si l'application en prévoit une.

Les clés étant des horodatages, relancer `load.py` dans la même heure réécrit les **mêmes clés** : pas de doublons. C'est un moyen simple de « rattraper » un nœud en retard.

Question : Que se passe-t-il lorsqu'un nœud devient indisponible ?

Réponse :Le Leader ne peut plus lui répliquer les données. La donnée envoyée pendant la panne est présente sur le Leader et le Follower 2, mais pas sur le Follower 1 : il a du retard. Selon l'application, le client voit une erreur, un timeout ou une réplication partielle.

Question : Les autres nœuds sont-ils toujours accessibles ?

Réponse : Oui. C'est une **panne partielle** : le système continue de fonctionner, avec moins de redondance.

Question : Quelle stratégie pourrait permettre à un système réel de continuer à fonctionner ?

Réponse : Répliquer sur plusieurs nœuds, accepter une écriture dès qu'une majorité (quorum) a confirmé, utiliser des timeouts et des retries, et élire un nouveau leader si c'est le Leader qui tombe.

Question : Que faudrait-il mettre en place pour récupérer automatiquement un nœud défaillant ?

Réponse : Détecter la panne (heartbeat, health check), redémarrer automatiquement le nœud (`restart: unless-stopped` dans Docker Compose, ou un orchestrateur comme Kubernetes), puis le resynchroniser pour qu'il rattrape les données manquées.

11. Étape 7 — Simulation d'une partition réseau

Ce qu'on observe	Explication
Le conteneur est toujours `Up` dans `docker ps`	Le nœud tourne, il est seulement injoignable.
Les 20 relevés sont absents sur le Follower 2	Le nœud d'entrée n'a pas pu le joindre pendant l'envoi.
Après `docker network connect`, le Follower 2 est joignable mais toujours en retard	Le réseau revient, mais les données manquées ne reviennent pas toutes seules.

Question : Quelle différence faites-vous entre panne d'un nœud et partition réseau ?

Réponse

- **Panne d'un nœud** : le nœud est arrêté, plus aucun processus ne tourne (`docker stop`).
- **Partition réseau** : le nœud tourne toujours, mais les autres ne peuvent plus communiquer avec lui.

Vu de l'extérieur, les deux ressemblent à un timeout : on ne peut pas savoir si le nœud est mort ou seulement coupé. La partition est plus dangereuse : le nœud isolé reste vivant et pourrait continuer à servir d'anciennes températures, voire accepter des écritures de son côté. Les copies **divergent** alors, et il faut choisir entre rester disponible (au risque de l'incohérence) ou refuser des requêtes pour rester cohérent : c'est le dilemme du théorème CAP.

12. Étape 8 — Analyse : les difficultés du distribué

Problème	Observation dans le TP	Conséquence
Panne d'un nœud	Follower 1 arrêté : il manque les relevés envoyés pendant la panne	Réplicas incomplets, moins de tolérance aux pannes

Partition réseau	Follower 2 déconnecté mais toujours `Up`, sans les relevés envoyés	Nœud injoignable, risque de lire des températures périmées
Synchronisation	Après reconnexion, le nœud reste en retard tant qu'on ne renvoie pas les données	Il faut un mécanisme de rattrapage
Réplication	Les 20 relevés envoyés au nœud d'entrée se retrouvent sur les autres nœuds	Plus de disponibilité, mais plus de messages et de stockage
Communication réseau	Le nœud d'entrée contacte les autres par leur nom sur distributed-net	Dépendance au réseau : latence, perte, coupure
Cohérence des données	Pendant la panne ou la partition, les nœuds n'ont pas les mêmes relevés	Réponses différentes selon le nœud interrogé

Question : Pourquoi distribuer les données et les traitements sur plusieurs nœuds ?

Réponse : Pour dépasser les limites d'un seul serveur (stockage, CPU, mémoire), répartir la charge, passer à l'échelle horizontalement et continuer à fonctionner malgré certaines pannes. Passer de 20 relevés de Paris à des milliers de villes sur plusieurs années ne tient plus sur une seule machine.

Question : Quels avantages apporte la réplication ?

Réponse : Haute disponibilité, tolérance aux pannes, durabilité des données (pas de perte si un nœud disparaît) et possibilité de répartir les lectures entre les copies.

Question : Quels problèmes supplémentaires apparaissent lorsque plusieurs nœuds doivent maintenir un état cohérent ?

Réponse : Garder les copies identiques, ordonner les écritures, gérer les conflits, supporter la latence de réplication, gérer les partitions réseau (CAP), élire un leader et éviter le *split-brain* (deux nœuds qui se croient leader). Dans le TP, le Follower 2 en retard illustre ce problème de cohérence.

Question : Pourquoi un système distribué est-il généralement plus complexe qu'un système centralisé ?

Réponse : Le réseau n'est pas fiable, les pannes sont partielles, il n'y a ni horloge ni état global communs, et il y a plus de composants à déployer, superviser et déboguer. Il faut aussi arbitrer en permanence entre cohérence, disponibilité et performance.